🧠 TRADING AI SYSTEM SPEC

1. Master Prompt

You are my AI trading copilot and senior software engineer. We are building a real-time SPY trading system together. This is not a simple bot — it is an always-on trading partner that helps me analyze markets, reason through trades, and make decisions during live market hours. SYSTEM GOAL: Build a Python-based trading engine that continuously tracks SPY across multiple timeframes and provides real-time reasoning, trade setup identification, and conversational decision support. CORE PRINCIPLES:

- The system must continuously track market data (seconds → minutes → hours → days)
- Use multi-timeframe analysis (1m, 5m, 15m, 1h, daily)
- Combine deterministic logic (Python) with AI reasoning (Claude)
- The system should support BOTH calls and puts (not long-only)
- I make final execution decisions — the system is a decision partner, not an auto trader ARCHITECTURE OVERVIEW:

1. Market Engine (Python)

- Ingest real-time SPY data (Alpaca)
- Build candles across multiple timeframes
- Compute indicators:
  - VWAP
  - EMA (9, 20, 50, 200)
  - RSI
  - MACD
  - ATR
- Detect:
  - trend direction
  - trend strength
  - trend weakening
  - trend flips
  - support/resistance
  - structure breaks

2. Strategy Logic

- Multi-timeframe confirmation:
  - Higher timeframe = trend bias
  - Lower timeframe = entry/exit timing
- Detect:
  - continuation setups
  - reversal setups
  - failed breakouts
  - breakdowns
- Trade both directions (calls + puts)

3. Trade Management

- Track active trades
- Exit conditions:
    - trend break
    - trend flip
    - profit target (percentage-based)
    - stop loss
- Support partial exits and runners

4. AI Layer (Claude)

- Interpret structured market state
- Explain what is happening in real time
- Answer questions like:
    - "Is this still a pullback or a reversal?"
    - "Calls or puts here?"
    - "Is momentum weakening?"
    - "Should I take profit?"
- Provide reasoning, not just signals

5. Event-Driven System

- The engine continuously tracks everything
- AI is triggered when:
    - meaningful changes occur
    - setups form or break
    - trade conditions change
    - I ask questions

6. Research Module (separate)

- Help find stocks with near-term upside
- Analyze themes (AI, space, etc.)
- Identify opportunities like RKLB IMPORTANT DESIGN RULE: Do NOT rely on AI for raw indicator calculations or core logic. Use Python for deterministic analysis. Use AI for reasoning, interpretation, and conversation. END GOAL: Create an always-on trading partner that:
- tracks SPY continuously
- reasons about trade setups in real time
- helps evaluate entries and exits
- communicates clearly and intelligently during market hours

Your role:

1. Help design and build this system step-by-step
2. Write clean, modular Python code
3. Ask clarifying questions when needed
4. Keep the system structured and scalable
5. Always align with the architecture above Let's start by designing the system architecture and folder structure, then begin with the market data + indicator engine.

Updated Prompt 2:

This is excellent. Before moving to Phase 2, I want to strengthen the system design:

1. Add a Market Narrative layer that summarizes the current dominant condition (trend, chop, transition, breakout, reversal).
2. Define precise rule-based logic for:
- entry confirmation (long and short)
- trend weakening
- trend break
- trend flip
- exit triggers (profit target, invalidation, structure loss)
3. Ensure TradeState (entry price, P&L, targets, stops, thesis) is included in MarketState and passed into the AI layer.
4. Define timeframe weighting (how 1m, 5m, 15m, 1h interact and which overrides).
5. Add a setup quality scoring system (0–100) based on alignment, momentum, and structure. Refine the architecture and then proceed to Phase 2.

Updated Prompt 3:

This is strong. Before moving further, revise the architecture with one additional design layer:

1. Add a hybrid local + hosted AI boundary:
- Local system on my MacBook handles market data ingestion, candle building, indicators, structure detection, scoring, event generation, logging, state persistence, and local memory/journal search.
- Hosted AI layer handles conversational reasoning, state interpretation, live trade discussion, news-aware commentary, and post-trade review.
2. Add a Context Intelligence Layer for external information:
- market news
- macro calendar/events
- Fed / CPI / jobs / earnings context
- headline importance scoring
- relevance to SPY and active trades
- structured summaries passed into the AI layer
3. Add source reliability rules:
- Tier 1: price/volume/official/macro/high-quality news
- Tier 2: curated research and my own journal/history
- Tier 3: optional low-trust crowd sentiment sources The system should not rely on Reddit or crowd ideas in the first serious version.
4. Update the architecture so the AI layer consumes:
- MarketState
- TradeState
- NarrativeState
- SetupScore breakdown
- ContextState (news/macro/headline summaries)
- MemoryContext (relevant prior trades, notes, patterns)
5. Design the system so that a local voice interface can be added later, with local speech-to-text and optional spoken responses, but do not make voice a core dependency yet. Please

revise the architecture diagram and module structure accordingly, but keep the current trading engine intact.

6. Keep the architecture extensible beyond trading:

- Trading is the first serious domain and current priority.
- However, I want the system designed so it can later expand into a broader personal AI assistant with additional modules/tools (for example productivity, media control, research, and other personal workflows).
- Do not broaden the implementation scope right now; just preserve extensibility in the architecture.

2. System Goal

- Real-time SPY trading copilot
- Multi-timeframe analysis
- Conversational decision partner
- Eventually extensible to personal AI (Jarvis-style)

3. Architecture

Layers:

- Market Engine (local)
- Indicator Engine
- Structure Detection
- Strategy Logic
- Trade Manager
- AI Layer (hosted)
- Event Bus
- Context Intelligence Layer (news/macro)
- Memory Layer (journal + history)

4. Hybrid AI Design

Local (MacBook)

- data ingestion
- indicators
- structure detection
- scoring
- event detection
- memory + logs

Hosted (Claude)

- reasoning

- conversation
- trade explanation
- news-aware insights

5. Timeframe Hierarchy

- Daily (40%) → macro context
- 1h (30%) → trend bias
- 15m (20%) → setup formation
- 5m (7%) → entry timing
- 1m (3%) → precision / stops

6. Strategy Logic

- multi-timeframe confirmation
- continuation + reversal setups
- calls + puts
- scoring system (0–100)

7. Trade Lifecycle

- Watching
- Setup identified
- Entry confirmed
- Active trade
- Exit (stop / flip / target / partial)

8. Key Design Decisions

- indicators = pure functions
- event-driven system
- MarketState → AI bridge
- narrative state machine
- scoring system (alignment, momentum, structure, timing)
- hybrid AI architecture
- news = structured input
- no Reddit in v1
- extensible beyond trading

9. Future Extensions

- voice interface
- personal assistant features
- local AI enhancements

- dashboard UI
- multi-symbol scanning